

# Learning to Explore Sample Relationships

Zhi Hou<sup>1</sup>, Baosheng Yu<sup>2</sup>, Chaoyue Wang<sup>3</sup>, Yibing Zhan<sup>3</sup>, and Dacheng Tao<sup>2</sup>, *Fellow, IEEE*

**Abstract**—Despite the great success achieved, deep learning technologies usually suffer from data scarcity issues in real-world applications, where existing methods mainly explore sample relationships in a vanilla way from the perspectives of either the input or the loss function. In this paper, we propose a batch transformer module, BatchFormerV1, to equip deep neural networks themselves with the abilities to explore sample relationships in a learnable way. Basically, the proposed method enables data collaboration, e.g., head-class samples will also contribute to the learning of tail classes. Considering that exploring instance-level relationships has very limited impacts on dense prediction, we generalize and refer to the proposed module as BatchFormerV2, which further enables exploring sample relationships for pixel-/patch-level dense representations. In addition, to address the train-test inconsistency where a mini-batch of data samples are *neither necessary nor desirable* during inference, we also devise a two-stream training pipeline, i.e., a shared model is first jointly optimized with and without BatchFormerV2 which is then removed during testing. The proposed module is plug-and-play without requiring any extra inference cost. Lastly, we evaluate the proposed method on over ten popular datasets, including 1) different data scarcity settings such as long-tailed recognition, zero-shot learning, domain generalization, and contrastive learning; and 2) different visual recognition tasks ranging from image classification to object detection and panoptic segmentation.

**Index Terms**—Deep representation learning, vision transformer, sample relationship.

## I. INTRODUCTION

RECENT success of deep learning heavily relies on collecting and annotating large-scale training data [1], [2], while data scarcity issues have been repeatedly found in real-world applications. Among the methods for handling data scarcity issues, sample relationship has received much attention from the community, especially for the tasks without proper training data distributions to guarantee good generalization performances, such as long-tailed recognition [3], zero-shot learning [4], and domain generalization [5]. An intuitive example

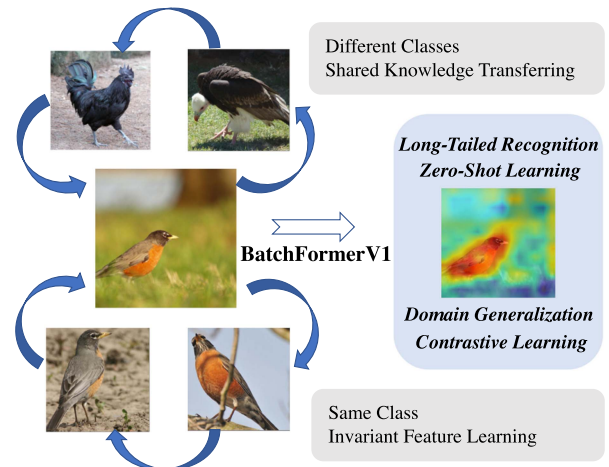


Fig. 1. An illustration of sample relationships. Specifically, similar classes tend to share some parts (e.g., cock, robin, vulture share body shape, and claw shape), and transferring the shared knowledge from head to tail classes thus facilitates learning with long-tailed distributions.

revealing the effectiveness of sample relationships is shown in Fig. 1.

Sample relationships have been intensively explored using an explicit scheme [6], [7], [8], [9], [10], [11], [12]. Specifically, a simple yet very effective way is to generate new samples from existing training data [8], such as mixup [7], copy-paste [9], crossgrad [6], and compositional learning [4], [13], [14], [15]. Another way is to transfer knowledge between different samples, e.g., 1) transferring the meta knowledge between head and tail classes for long-tailed recognition [10], [16]; 2) transferring the knowledge from seen to unseen classes for zero-shot learning [4], [17]; and 3) transferring the invariant knowledge for domain generalization [18], [19], [20]. However, all above-mentioned methods explore sample relationships from either the input or the loss function, failing to enable deep neural networks themselves with the abilities to automatically explore sample relationships, i.e., no interaction appears in the batch dimension during representation learning.

The transformer architecture has been very widely used to explore spatial-temporal relationships [21], [22], [23], [24], [25], [26], while it is non-trivial to apply attention mechanism in the batch dimension due to the train-test inconsistency [12], [16], [27]. For example, a mini-batch of data samples is neither necessary nor desirable during inference, while it always requires to track training statistics as the running mean and variance when applying the batch normalization [27]. Another example keeps all category centers during training, which is

Received 15 September 2022; revised 30 December 2024; accepted 22 February 2025. Date of publication 10 March 2025; date of current version 6 June 2025. This work was supported by the National Research Foundation, Singapore under Grant NRF-P2024-001, and in part by the LKCMedicine Start-Up Grant (Dr Baosheng Yu). Recommended for acceptance by M.-L. Zhang. (Corresponding authors: Baosheng Yu; Dacheng Tao.)

Zhi Hou is with the School of Computer Science, University of Sydney, Sydney, NSW 2050, Australia (e-mail: zhou9878@uni.sydney.edu.au).

Baosheng Yu is with the Lee Kong Chian School of Medicine, Nanyang Technological University, Singapore 639798 (e-mail: baosheng.yu@ntu.edu.sg).

Chaoyue Wang and Yibing Zhan are with JD Explore Academy, Beijing 100176, China (e-mail: chaoyue.wang@outlook.com; zhanyibing@jd.com).

Dacheng Tao is with the College of Computing and Data Science, Nanyang Technological University, Singapore 639798 (e-mail: dacheng.tao@ntu.edu.sg).

Code is available at <https://zhihou7.github.io/BatchFormer>.

This article has supplementary downloadable material available at <https://doi.org/10.1109/TPAMI.2025.3549300>, provided by the authors.

Digital Object Identifier 10.1109/TPAMI.2025.3549300

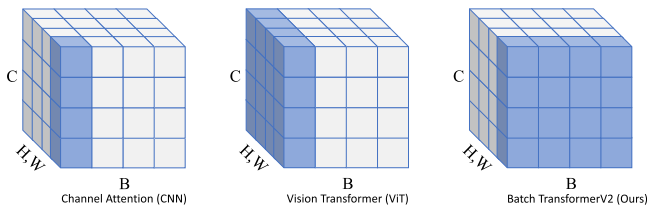


Fig. 2. An illustration of the attention mechanisms on channel, spatial, and batch dimensions.

then used to enhance the tail/unseen categories during testing [12], [16]. To empower deep neural networks with structural advances for sample relationship learning, we introduce the attention mechanism to the batch dimension and address the train-test inconsistency as follows. Specifically, we capture the sample relationships within each mini-batch of training data by applying a transformer encoder into the batch dimension, and refer to it as Batch Transformer or BatchFormerV1. An intuitive example showing the differences between the channel, spatial, and batch attentions is illustrated in Fig. 2. To address the train-test inconsistency, we then utilize a shared classifier before and after the proposed BatchFormerV1 module to achieve the batch-invariant learning, i.e., learn representations invariant to with and without mini-batch. By doing this, BatchFormerV1 is only required during training, and we do not need to change the inference structure of the original model.

Inspired by BatchFormerV1 for instance-level visual recognition, we further propose to enable exploring the sample relationships for pixel-/patch-level dense representation learning. That is, exploring sample relationships with only instance-level representations that are usually from the last fully connected layer has very limited influences on dense representation learning. Therefore, we generalize BatchFormerV1 such that it can be used in different intermediate layers to explore semantics at different scales, ranging from the pixel to the patch and the whole image. Nevertheless, it is difficult to setup a shared classifier before and after each layer for the batch-invariant learning. We thus introduce a generalized module, BatchFormerV2, to enable the information propagation between different data samples at multiple semantic levels by learning the batch-invariant representations. By doing this, the proposed BatchFormerV2 module can be applied in different intermediate layers of popular vision transformers. The train-test inconsistency is addressed by a two-stream training pipeline, i.e., one stream with BatchFormerV2 and the other stream without, while all other layers are shared by both streams. All shared layers will be jointly optimized to generalize on the inputs with and without BatchFormerV2. During testing, we can directly remove all BatchFormerV2 modules without sacrificing model performance.

During training, the proposed method enables the information propagation of all samples in each mini-batch. Therefore, all samples can contribute to the learning on each categories. Intuitively, it implicitly enriches current training samples with hallucinated features from the whole mini-batch (e.g., the shared

parts between two categories). For example, in long-tailed recognition, such hallucinated features can also improve the feature space of the tail classes. Meanwhile, the loss function also emphasizes on the rare classes via propagating larger gradients of rare classes in each mini-batch. In addition, we also find two obvious changes on the learned representations with the proposed module, i.e., it effectively facilitates the model to learn 1) comprehensive representations by focusing on almost all different object parts; and 2) invariant representations by focusing on the object itself rather than complex backgrounds (See more results in Section III-F, Section IV-E, and Appendix).

A preliminary version of BatchFormerV1 has been published in [28] on the IEEE/CVF Computer Vision and Pattern Recognition Conference (CVPR2022). The description of BatchFormerV2 also appears in the preprint [29], which is *not* submitted to any other conference or journal. Our main contributions can be summarized as follows:

- We propose to learn sample relationships from the perspective of improving the internal structure of deep neural networks during training.
- We devise a simple yet effective module termed as BatchFormerV1 to explore sample relationships for different data scarcity tasks.
- We introduce a generalized BatchFormerV2 to enable sample relationship learning for dense prediction.
- We develop a two-stream training strategy to address the train-test inconsistency, where mini-batch samples are not necessary during the test phase, ensuring that the proposed module can improve model performance without any extra inference cost.
- We perform extensive experiments in different settings and tasks, which show the effectiveness of the proposed method, including long-tailed recognition, zero-shot learning, domain generalization, self-supervised representation learning, image classification, object detection, and panoptic segmentation.

## II. RELATED WORK

*Sample Relationship:* There are rich relationships among different samples, which have been widely used via various strategies [7], [30], [31], [32]. Specifically, Zhang et al. [7] propose to regularize the model to favor simple linear behavior in-between training samples with mixup. However, mixup [7] merely considers a linear transformation between data samples, while we aim to investigate the non-linear relationship among samples in a more powerful way. The compositionality of samples has also inspired many approaches to improve few/zero-shot generalization [4], [14], [33], [34], where the parts/attributes shared among different samples have been explored via the prior knowledge on label relationships. Several approaches also use sample/class relationships to conduct transductive inference [16], [30], [31], [32], e.g., transductive few-shot classification [30], meta embedding [12], [16], and non-parametric transformer [35]. However, those approaches usually require to inference with multiple samples (e.g., query set, or bank features). Meanwhile, many recent domain generalization methods [18], [19], [20] aim to

find casual/invariant representations across domains, which we think them internally utilize the relationship among samples of the same class but different domains.

*Data Scarcity Learning:* Learning with limited and imperfect data has turned out to be very challenging in a variety of data scarcity tasks. For example, in many real-world applications, the data from different classes usually follow a long-tailed distribution where a large portion of classes have very few instances. Current long-tailed learning approaches can be roughly categorized into 1) distribution re-balancing (e.g., re-sampling [36], [37], [38], re-weighting [39], [40], [41], [42], [43]); 2) ensemble of diverse experts [3], [44], [45]; and 3) knowledge transfer [10], [11], [12], [16], [46]. Other data scarcity tasks include zero-shot learning and domain generalization [47], [48], [49]. Specifically, zero-shot learning aims to recognize unseen classes without training samples, while domain generalization aims at generalizing from seen to unseen domains. Current zero-shot learning approaches [17], [50], [51], [52] usually transfer knowledge of seen classes to unseen classes via modern techniques (e.g., graph network [4], data generalization [53], and compositional learning [4], [14]). Recently, compositional zero-shot learning has been widely explored in different tasks [4], [13], [14], [15], and we thus mainly evaluate the proposed module on compositional zero-shot learning [4]. In addition, recent domain generalization methods usually include data augmentation [48], [54], meta learning [55], [56], and disentangled/invariant representation learning [18], [19], [57], [58]. The proposed method facilitates robust representation learning without any assumptions on the dataset distributions and can thus be used for different tasks and datasets.

*Vision Transformer:* Transformer was first introduced by Vaswani et al. [21] for machine translation. As a core part, attention mechanism aggregates information from the entire input sequence and then update it [59]. In the past few years, transformer-based architectures have dominated in natural language processing (NLP), e.g., BERT [60] has shown superior performance among massive downstream NLP tasks. Besides, self-attention mechanism also demonstrates powerful modeling ability for non-grid data [61], and thus improves graph representation learning. In addition, transformer models have presented a new paradigm for computer vision tasks, including classification [22], [23], detection [62], [63], [64], [65], segmentation [62], [66], [67], [68], [69], [70], and representation learning [71], [72], [73]. Specifically, Dosovitskiy et al. [22] introduced a pure vision transformer model, termed as ViT, by applying a sequence of image patches, which achieves impressive performance on image classification. Recently, vision transformer [22], [23] has gradually become the new backbone for vision tasks, and massive large models based on transformers have emerged in computer vision, including CLIP [74], MoCo [75], DALL-E [76], and MAE [73]. Except for the backbone, transformer-based models, e.g., DETR [62], have also reformed the pipeline of detection and segmentation. Specifically, DETR [62], constructed upon the encoder-decoder transformer architecture, demonstrates a clear set-based method for detection and greatly simplifies the traditional pipeline which includes many hand-designed components. Recently,

Zhu et al. [63] present Deformable-DETR, which largely accelerates the convergence and improves the performance. Though the great success of transformer in computer vision, current approaches merely investigate the spatial self-attention, while ignoring the pixel/patch-level relationships among different data samples.

### III. METHOD

In this section, we first provide an overview of the main deep representation learning framework with BatchFormerV1. We then review vanilla vision transformer architecture and introduce BatchFormerV1 in detail. After that, we present a generalized BatchFormerV2 with the two-stream training strategy. Lastly, we provide some insights and give a discussion on the proposed method.

#### A. Overview

We aim to enable deep neural networks themselves with the ability to learn the relationships from a mini-batch of data samples during the end-to-end deep representation learning. The main deep representation learning framework with BatchFormerV1 is shown in Fig. 3. Specifically, a backbone network is first used to learn representations for individual data samples, i.e., there is no interaction between different samples in each mini-batch. After this, a new module is introduced to capture the relationships between different samples by using the cross-attention mechanism in transformer, and we refer to it as Batch Transformer or BatchFormerV1. The output of BatchFormerV1 is then used as the input of the final classifier. To address the train-test inconsistency, where mini-batch samples are not required during the test phase, we use an auxiliary classifier before BatchFormerV1. By sharing weights with the final classifier, this approach enables the transfer of knowledge about sample relationships to both the backbone and the auxiliary classifier. During testing, we can thus remove BatchFormerV1 and directly use the auxiliary classifier for classification.

#### B. Revisiting Vision Transformer

The great success of transformer architecture in NLP has recently spread to almost every region of computer vision known as vision transformers [22], [23], [62]. The transformer architecture not only achieves superior performance in different vision tasks [23] (e.g., image classification, object detection, and semantic segmentation), but also brings novel paradigms for some fundamental tasks (e.g., DETR [62] for object detection and MAE [73] for self-supervised learning). Among typical vision Transformers, the overall model usually consists of a stack of multiple Transformer encoder blocks [21], where each transformer encoder block contains a multi-head self-attention layer (MSA) followed by a feed-forward network (FFN). Specifically, the self-attention mechanism used in vision Transformer can be described as follows. Given  $\mathbf{Q}$ ,  $\mathbf{K}$ ,  $\mathbf{V} \in \mathbb{R}^{N \times C}$  as the query, key, and value, respectively, where  $N$  is the number of image patches (or tokens) and  $C$  is the embedding dimension. We then



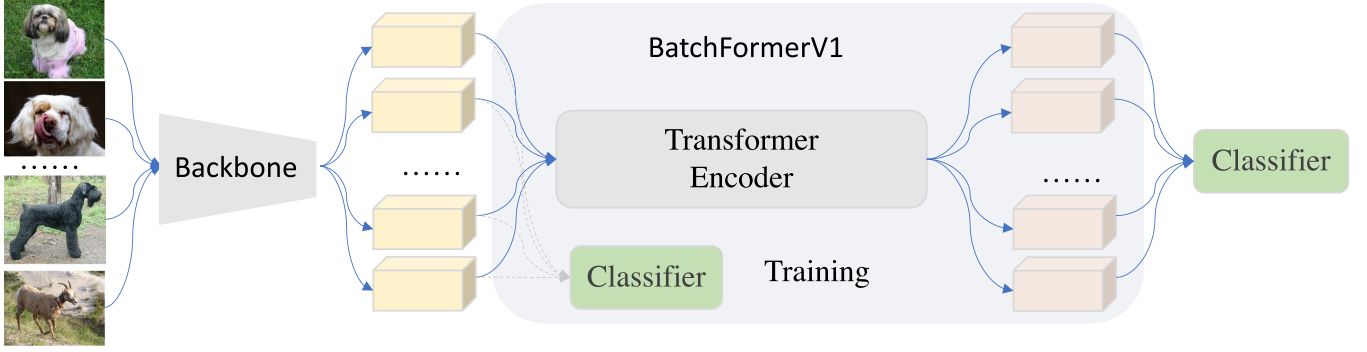


Fig. 3. The main deep representation learning framework with BatchFormerV1. Specifically, we apply BatchFormerV1 between the backbone network (e.g., ResNet) and the classifier to explore sample relationships. With a shared classifier for training, we can remove BatchFormerV1 during testing.

have the output  $\mathbf{Z}$  for the self-attention module:

$$\mathbf{Z} = \text{softmax} \left( \frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{C}} \right) \mathbf{V}, \quad (1)$$

where  $\mathbf{Q}$ ,  $\mathbf{K}$ , and  $\mathbf{V}$  are learned from the same input. Specifically, multi-head self-attention module applies attention by splitting the input into multiple representation subspaces and then concatenates the representations from different heads. From a perspective of information propagation, the transformer architecture aggregates the feature of the tokens via spatial attention. Different from attention layers in a typical vision transformer, the proposed batch transformer layer performs a standard self-attention operation along the batch dimension. This operation is not conditioned on the class information, and the sample relationships are learned in an end-to-end manner. As a result, the method can be applied in self-supervised, semi-supervised, and supervised settings. Notably, the term “sample relationship” refers to how the representation learning of each sample is influenced by other samples (or image parts, regions), as captured by the attention matrix.

### C. BatchFormerV1

*Transformer Encoder:* The transformer encoder includes multi-head self-attention (MSA) and MLP blocks. A LayerNorm (LN) is used after each block. Let  $\mathbf{X} \in \mathbb{R}^{N \times C}$  denote a sequence of input features, where  $N$  is the length of the sequence and  $C$  indicates the dimension of input features. We then have the output of the transformer encoder as follows,

$$\hat{\mathbf{X}}_l = \text{LN}(\text{MSA}(\mathbf{X}_{l-1}) + \mathbf{X}_{l-1}), \quad (2)$$

$$\mathbf{X}_l = \text{LN}(\text{MLP}(\hat{\mathbf{X}}_l) + \hat{\mathbf{X}}_l), \quad (3)$$

where  $l$  indicates the index of layers in the transformer encoder. The multi-head self-attention layers have been widely used to model the relationships from channel and spatial dimensions [21], [22], [77]. Therefore, we argue that it can also be extended to explore the relationships in the batch dimension. As a result, different from typical usage of transformer layers, the input of BatchFormerV1 will be first reshaped to enable the transformer layers working on the batch dimension of the input

```
def batchformer_v1(x, y, encoder, is_training):
    # encoder: TransformerEncoderLayer(C, 4, C)
    if not is_training:
        return x, y
    orig_x = x
    x = encoder(x.unsqueeze(1)).squeeze(1)
    x = torch.cat([orig_x, x], dim=0)
    y = torch.cat([y, y], dim=0)
    return x, y
```

Fig. 4. Python code of BatchFormerV1 based on PyTorch.

data. By doing this, the self-attention mechanism in transformer layers then becomes the cross-attention between different samples for BatchFormerV1.

*Shared Classifier:* Since we can not assume batch statistics for testing, such as sample relationships, there might be a gap between the features before and after the BatchFormerV1 module. That is, we can not perform inference on new samples by directly removing BatchFormerV1. Therefore, apart from the final classifier, we also introduce a new auxiliary classifier to not only learn from the final classifier but also keep consistent with the features before BatchFormerV1. To achieve this, we simply share the parameters/weights between the auxiliary classifier and the final classifier. We refer to this simple yet effective strategy as “shared classifier”. With the proposed “shared classifier”, we can thus remove BatchFormerV1 during testing. BatchFormerV1 is thus a plug-and-play module for robust deep representation learning. BatchFormerV1 is also easy to implement using typical deep learning packages, e.g., Fig. 4 shows how to implement BatchFormerV1 with several lines of python code based on Pytorch [78].

### D. BatchFormerV2

To generalize the batch attention mechanism into pixel/patch level feature maps for dense representation learning, we devise BatchFormerV2 as follows. Given  $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{B \times N \times C}$ , we then have

$$\mathbf{Z}_i = \text{softmax} \left( \frac{\mathbf{Q}_i \mathbf{K}_i^T}{\sqrt{C}} \right) \mathbf{V}_i, \quad \mathbf{Z} = \text{concat}(\mathbf{Z}_1, \dots, \mathbf{Z}_N), \quad (4)$$

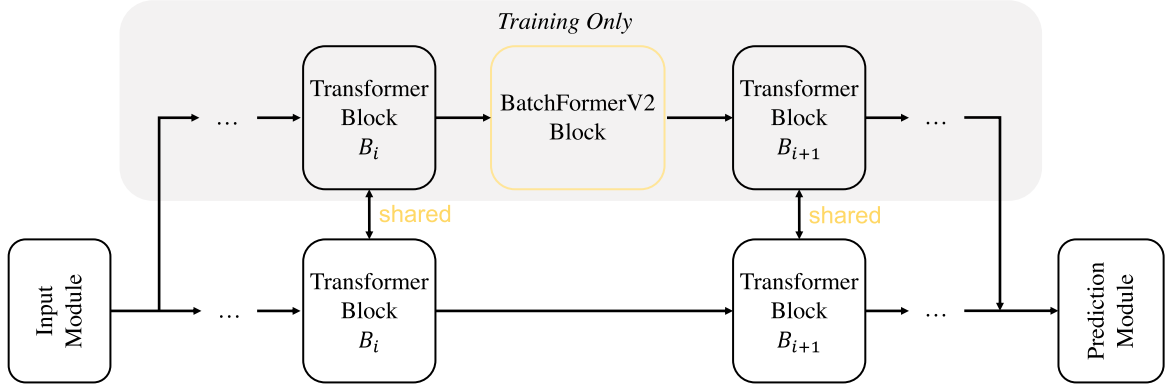


Fig. 5. The two-stream training pipeline for BatchFormerV2. For example, the input indicates the feature map from backbone for DETR [62], while the input is the feature map after the patch embedding layer for ViT [22]. The outputs of two streams are the input of the shared prediction module. In addition, the transformer blocks and the prediction module, e.g., the transformer decoder in DETR [62] and the classification head in ViT [22], are shared by two streams. During inference, the stream with BatchFormerV2 is removed.

```
def batchformer_v2(x, encoder, is_training, is_first):
    # x: input with the shape (B, N, C).
    # encoder: TransformerEncoderLayer(C, nhead, C,
    # batch_first=False)
    if not is_training:
        return x
    orig_x = x
    if not is_first:
        orig_x, x = torch.split(x, len(x)//2)
    x = encoder(x)
    x = torch.cat([orig_x, x], dim=0)
    return x
```

Fig. 6. Python code of BatchFormerV2 based on PyTorch.

where  $\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i \in \mathbb{R}^{B \times C}$  and  $\mathbf{Z} \in \mathbb{R}^{B \times N \times C}$ . Given the input for a specific layer/block with the spatial dimensions  $H, W$ , i.e., the number of image patches is  $N = H \times W$ . During training, at each spatial position  $i = 1, \dots, N$ , we treat the batch of patch features in current position as a sequence, i.e., we have  $N$  sequences each with the length of  $B$ . All above-mentioned sequences are then fed into a shared transformer block. Fig. 5 illustrates the two-stream training pipeline for BatchFormerV2.

Two reasons that we use a shared transformer block are as follows: 1) it will increase the computation and memory consumption considerably if we use different blocks at different spatial positions; 2) it will be difficult to dense prediction with different sizes of input images, which is in line with the motivation of fully convolutional networks (FCNs) for dense prediction [79] as well as the convolution operations [80] and channel-wise attentions [81]. Therefore, we share the transformer block among the spatial dimensions in BatchFormerV2. By doing this, the proposed BatchFormerV2 can be implemented by simply transposing the spatial and batch dimensions before the standard multi-head self-attention layer. Noticeably, as illustrated in Fig. 6, BatchFormerV2 can also be easily implemented with a few lines of code using popular deep learning packages such as PyTorch [78].

### E. Two-Stream Training

The main challenge of batch operations in representation learning is train-test inconsistency. We address this issue in BatchFormerV1 by introducing a shared classifier. Inspired by

this, we further generalize this solution to dense representation learning, i.e., learning batch-invariant representations. To achieve this, we introduce a new two-stream training strategy as follows: When applying BatchFormerV2, we create a new siamese stream, followed by BatchFormerV2, while keeping the original stream unchanged. Notably, all transformer blocks are shared by both streams during training. This means that the shared blocks are trained on a mixture of distributions, whether or not BatchFormerV2 is included. Therefore, during testing, the original stream performs well whether or not a mini-batch of testing data is available. To avoid introducing any extra inference load, we remove BatchFormerV2 for testing. Lastly, with the proposed two-stream training strategy, BatchFormerV2 can be easily integrated into existing applications using transformer architectures in a plug-and-play manner, such as ViT [22] for image classification and DETR [62]/Deformable-DETR [63] for object detection.

### F. Discussion

To better understand how the proposed method helps representation learning by exploring sample relationships, we also provide an intuitive explanation from the perspective of gradient propagation for optimization. For simplicity, we use BatchFormerV1 as an example. Intuitively, without BatchFormerV1, all losses only propagate gradients on the corresponding samples and categories, i.e., one-to-one, while there are gradients on other samples with BatchFormerV1 (the dashed line) as illustrated in Fig. 7. Specifically, given samples  $X(X = X_0, X_1, X_i, \dots, X_{N-1})$  and the corresponding losses  $L_0, L_1, L_i, \dots, L_{N-1}$  in the mini-batch, with BatchFormerV1, we then have

$$\frac{\partial L_i}{\partial X} = \frac{\partial L_i}{\partial X_i} + \sum_{j \neq i}^{N-1} \frac{\partial L_i}{\partial X_j}. \quad (5)$$

That is, BatchFormerV1 brings new gradient terms  $\frac{\partial L_i}{\partial X_j}$ , where  $i \neq j$ . From a perspective of gradient optimization,  $L_i$  also optimizes the network according to sample  $X_j (j \neq i)$ , that is

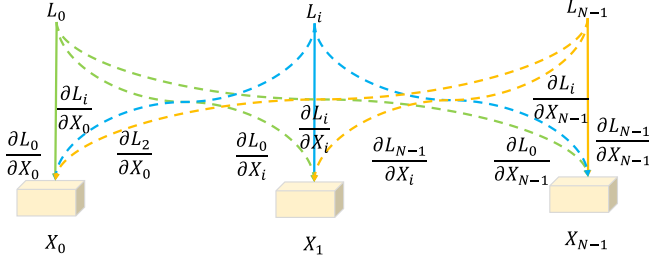


Fig. 7. The gradient propagation scheme with the proposed BatchFormerV1 module. Dashed lines represent the new gradient propagation among data samples.

a significant difference compared to the model without BatchFormerV1. In other word,  $X_j (j \neq i)$  can be regarded as a virtual sample [7], [46] of  $y_i$ , where  $y_i$  is the label of  $X_i$ . We consider that both BatchFormerV1 and Mixup [7] can be regarded as data-dependent augmentations. BatchFormerV1 implicitly draws virtual examples [7] from the vicinity distribution of samples via cross-attention module. From this perspective, BatchFormerV1 has implicitly augmented  $N - 1$  virtual samples for each label  $y_i$  via the relationship modeling among samples in the mini-batch. Previous approaches [53], [55] have demonstrated data augmentation is helpful for long-tailed recognition [11], [46], zero-shot learning [53], and domain generalization [48], [54]. Our gradients analysis in Section IV-E also demonstrates the tail classes have larger gradients on other samples compared to head classes.

#### IV. EXPERIMENTS

In this section, we perform extensive experiments to show the effectiveness of the proposed method in 1) a variety of data scarcity learning settings, including long-tailed recognition, zero-shot learning, domain generalization, and contrastive learning; and 2) different visual recognition tasks, including image classification, object detection, and panoptic segmentation. For simplicity, we use BFV1 for BatchFormerV1 and BFV2 for BatchFormerV2 in all Tables.

##### A. Long-Tailed Recognition

In this subsection, we evaluate BatchFormerV1 on long-tailed recognition by using recent state-of-the-art methods, Balanced-Softmax [82], RIDE [3], and PaCo [83], as our baseline. We use four popular datasets as follows: 1) CIFAR-100-LT has 50,000 training images and 10,000 validation images with 100 categories; 2) ImageNet-LT [16] contains 115.8 K images of 1000 classes from ImageNet2012. The number of images in each class ranges from 5 to 1,280; 3) iNaturalist2018 [84] is a large-scale fine-grained dataset with 437.5 K images from 8,142 categories; and 4) Places-LT [16] is a long-tailed scene classification dataset derived from [85] with 184.5 K images from 365 categories with the number of per-category images ranging from 5 to 4,980. If not otherwise stated, we follow the same settings used in baseline methods. Particularly, there is a small difference for RIDE [3]. We train the model with the

TABLE I  
ILLUSTRATION OF IMBALANCE RATIO 100 AND 200 ON CIFAR-100-LT

| Method         | 100         |             |             |             | 200         |             |             |             |
|----------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|
|                | All         | Many        | Med         | Few         | All         | Many        | Med         | Few         |
| RIDE [3]       | 48.0        | 68.1        | 49.2        | 23.9        | -           | -           | -           | -           |
| Balanced [83]* | 50.7        | 68.0        | 49.7        | 31.9        | 46.4        | 70.0        | 51.5        | 24.3        |
| + BFV1         | <b>51.7</b> | <b>68.4</b> | <b>49.3</b> | <b>34.3</b> | <b>47.5</b> | <b>70.2</b> | <b>53.3</b> | <b>25.5</b> |
| PaCo [84]      | 51.9        | 63.9        | <b>53.0</b> | <b>36.5</b> | 47.1        | 68.1        | 51.5        | 27.5        |
| + BFV1         | <b>52.4</b> | <b>68.4</b> | 52.1        | 34.0        | <b>47.8</b> | 68.1        | <b>53.0</b> | <b>28.2</b> |

\*indicates the model is trained with the official code in one stage (e.g., [82]).

TABLE II  
ILLUSTRATION OF RESNET-10/50 ON IMAGENET-LT

| Method         | ResNet-10   |             |             |             | ResNet-50   |             |             |             |
|----------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|
|                | All         | Many        | Med         | Few         | All         | Many        | Med         | Few         |
| OLTR [16]      | 35.6        | 43.2        | 35.1        | 18.5        | -           | -           | -           | -           |
| LFME [87]      | 38.8        | 47.0        | 37.9        | 19.2        | -           | -           | -           | -           |
| Balanced [83]* | 41.0        | 52.6        | 38.3        | 18.0        | 50.1        | 61.1        | 47.5        | 27.6        |
| + BFV1         | <b>43.2</b> | <b>52.8</b> | <b>40.7</b> | <b>24.9</b> | <b>51.1</b> | <b>61.4</b> | <b>47.8</b> | <b>33.6</b> |
| RIDE-3e [3]*   | 44.7        | 57.0        | 40.3        | 25.5        | 53.6        | 64.9        | 50.4        | 33.2        |
| + BFV1         | <b>45.7</b> | 56.3        | <b>42.1</b> | <b>28.3</b> | <b>54.1</b> | 64.3        | <b>51.4</b> | <b>35.1</b> |
| PaCo [84]      | -           | -           | -           | -           | 57.0        | 64.8        | 55.9        | 39.1        |
| + BFV1         | -           | -           | -           | -           | <b>57.4</b> | 62.7        | <b>56.7</b> | <b>42.1</b> |

two stage

|             |             |      |             |             |             |      |             |             |
|-------------|-------------|------|-------------|-------------|-------------|------|-------------|-------------|
| RIDE-3e [3] | 45.9        | 57.6 | 41.7        | 28.0        | 54.9        | 66.2 | 51.7        | 34.9        |
| + BFV1      | <b>47.6</b> | 55.3 | <b>45.5</b> | <b>33.3</b> | <b>55.7</b> | 64.6 | <b>53.4</b> | <b>39.0</b> |

\*indicates the model is trained with released code in one stage. RIDE-3e indicates three experts used in RIDE.

batch size 400 on four V100 GPUs for 100 epochs with an initial learning rate of 0.1 and 0.2 on ImageNet-LT and iNaturalist2018, respectively. The learning rate is decayed with cosine schedule on iNaturalist2018.

**Results on CIFAR-100-LT:** Table I shows that the proposed BatchFormerV1 is orthogonal to recent state-of-the-art methods such as Balanced-Softmax [82] and PaCo [83]. We notice that BatchFormerV1 improves Balanced-Softmax by 2.4% for Few classes when the imbalance ratio is 100, and by 1.8% on Medium classes and by 1.2% on Few classes, respectively, when the imbalance ratio is 200. Besides, the performance of PaCo on imbalance ratio 200 also increases by 1.5% on Medium classes and 0.7% on Few classes, respectively. For ratio 100, BatchFormerV1 mainly improves Many classes since PaCo has achieved good performance on Few classes. Overall, BatchFormerV1 improves the recognition of tail classes while maintaining the performance of head classes.

**Results on ImageNet-LT:** In Table II, BatchFormerV1 improves Balanced-Softmax by 2.4% on medium classes and **6.9%** on few classes respectively. Meanwhile, when using a ResNet-50 backbone, BatchFormerV1 improves Balanced-Softmax on the Few category by **5%**. When BatchFormerV1 is applied in RIDE [3], the results on Medium classes and Few classes increase by 1.8% and 2.8%, respectively. With a ResNet-50 backbone, BatchFormerV1 also improves Medium and Few classes by 1% and 1.9%, respectively. BatchFormerV1 achieves clear improvement overall, while the performance on Many classes drops. Furthermore, BatchFormerV1 also effectively improves RIDE under two-stage training strategy (RIDE uses a larger model to teach small model with distilling loss). Here, different from RIDE, we use a pre-trained model (the same model) to initialize the model and train the model again with BatchFormerV1. PaCo [83] is recently introduced for long-tailed recognition with supervised contrastive learning. We also find

TABLE III  
ABLATION STUDIES OF BATCH SIZE. THE BACKBONE IS RESNET-10

|        | B=16        | B=32        | B=64        | B=128 | B=256 | B=512       |
|--------|-------------|-------------|-------------|-------|-------|-------------|
| All    | 43.2        | <b>43.6</b> | 43.3        | 43.2  | 42.4  | 42.5        |
| Many   | <b>53.8</b> | 53.7        | 52.9        | 52.8  | 52.2  | 52.0        |
| Medium | 40.2        | 40.7        | <b>40.8</b> | 40.7  | 39.5  | 39.6        |
| Few    | 23.9        | 24.9        | 25.3        | 24.9  | 25.2  | <b>25.8</b> |

The backbone is ResNet-10.

TABLE IV  
ABLATION STUDIES ON THE SHARED CLASSIFIER

|                               | All         | Many        | Medium      | Few         |
|-------------------------------|-------------|-------------|-------------|-------------|
| Balanced [83] + BFV1 (shared) | <b>50.9</b> | <b>60.7</b> | <b>47.7</b> | 34.1        |
| Balanced [83] + BFV1          | 42.4        | 41.3        | 43.3        | <b>42.2</b> |

The backbone is ResNet-50.

TABLE V  
RESULTS ON INATURALIST2018

| Method       | All         | Many | Medium      | Few         |
|--------------|-------------|------|-------------|-------------|
| BBN [44]     | 66.3        | 49.4 | 70.8        | 65.3        |
| cRT [88]     | 65.2        | 69.0 | 66.0        | 63.2        |
| RIDE-3e [3]  | 72.2        | 70.2 | 72.2        | 72.7        |
| PaCo [84]    | 73.2        | -    | -           | -           |
| RIDE-3e [3]* | 72.5        | 68.1 | 72.7        | 73.2        |
| + BFV1       | <b>74.1</b> | 65.5 | <b>74.5</b> | <b>75.8</b> |

BatchFormerV1 can facilitate ImageNet-LT on Medium classes and Few classes. Noticeably, PaCo uses a strong data augmentation strategy from supervised contrastive learning with 400 training epochs. We consider that data augmentation degrades the improvement of BatchFormerV1 on ImageNet-LT. We also show that BatchFormerV1 achieves comparable results on balanced ImageNet in the appendix.

In addition, we show the influences of different batch size and the shared classifier in Tables III and IV, respectively. Specifically, in Table III, we find that BatchFormerV1 is less sensitive to the batch size that is small than 128, while the best performance on Few category is achieved with the batch size 512; In Table IV, we see that BatchFormerV1 without a shared classifier achieves the similar performance on three categories, while BatchFormerV1 with a shared classifier maintains the performance on Many category. This demonstrates the effectiveness of a shared classifier and the re-balancing ability of BatchFormerV1.

*Results on iNaturalist2018:* We only evaluate BatchFormerV1 on RIDE since Balanced-Softmax has very limited performance on iNaturalist2018 while training PaCo requires over 36 GPU days. We train RIDE with cosine decay learning-rate scheduler in one stage and obtain a baseline slightly better than reported with 72.5%. As illustrated in Table V, BatchFormerV1 improves the Medium and Few by 1.8% and 2.6%, respectively.

*Results on Places-LT:* Table VI illustrates that BatchFormerV1 improves BALMS on the Few category. Besides, BatchFormerV1 effectively improves PaCo [83], which also delivers a new state-of-the-art on Places-LT. Different from the results on CIFAR-100-LT and ImageNet-LT, here BatchFormerV1 mainly improves the Many category, which is possibly because PaCo has much worse performance on the Many

TABLE VI  
RESULTS ON PLACES-LT

| Method                  | All         | Many        | Medium      | Few         |
|-------------------------|-------------|-------------|-------------|-------------|
| OLTR [16]               | 35.9        | 44.7        | 37.0        | 25.3        |
| $\tau$ -normalized [88] | 37.9        | 37.8        | 40.7        | 31.8        |
| BALMS [83]              | 37.8        | 41.4        | 38.8        | 29.1        |
| + BFV1                  | <b>38.2</b> | 39.5        | 38.3        | <b>35.7</b> |
| PaCo [84]               | 41.2        | 37.5        | <b>47.2</b> | 33.9        |
| + BFV1                  | <b>41.6</b> | <b>44.0</b> | 43.1        | 33.7        |

The backbone is ResNet-152.

category and BatchFormerV1 can re-balance the learning on imbalanced training data.

### B. Zero-Shot Learning

We evaluate BatchFormerV1 for compositional zero-shot learning on three popular datasets: 1) MIT-States [89] consists of 30,000 training images of natural objects with 1,262 seen compositions (23.8 image per composition, 115 states and 245 objects), and 13,000 test images with 400 seen compositions and 400 unseen compositions; 2) UT-Zappos [90] includes 23,000 training images of shoes catalogue and we use the splits from [91]. UT-Zappos has 83 seen compositions for training (277.1 images per composition, 16 states, 12 objects) and 18 seen compositions and unseen compositions in test set; 3) C-GQA [4] provides 26,000 training images with 6,963 seen compositions (3.7 images per composition, 453 states, 870 objects) and 3,000 test images with 18 seen compositions and unseen compositions. All experiments are evaluated following the same settings in [4]. We adopt the evaluation protocol of [91] and report the Area Under the Curve (AUC) (in %) between the accuracy on seen and unseen compositions. Similar to [4], we also report unseen accuracy, seen accuracy, and the best harmonic mean.

*Results:* Table VII shows that BatchFormerV1 effectively improves the AUC and HM among all datasets compared to the baseline. For fair comparison, we use the official code and the same setting to reproduce [4] as our baseline. We notice that BatchFormerV1 mainly improves the Seen category on MIT-States and C-GQA, while it largely improves the Unseen category by nearly 5% on UT-Zap50 K. This might be because the number of seen composition instances on MIT-States and C-GQA is few, e.g. 23.8 image per seen composition on MIT-States and 3.7 images per seen composition. In other words, the recognition of seen compositions on MIT-States and C-GQA is few-shot learning. We think BatchFormerV1 can find invariant features among images of the same class on two datasets.

### C. Domain Generalization

In this subsection, we first show that BatchFormerV1 effectively improves the baseline without other domain generalization techniques on PACS [92]. We then apply BatchFormerV1 to a recent popular domain generalization method, e.g., SWAD [5].

We perform experiments on four popular domain generalization datasets: 1) PACS [92] covers 7 object categories and 4 domains (Photo, Art Paintings, Cartoon and Sketches); 2) VLCS [93] contains 10,729 images from 4 domains and 5 classes; 3) OfficeHome [94] contains 15,588 images from 4



TABLE VII  
RESULTS ON COMPOSITIONAL ZERO-SHOT LEARNING

| Method       | MIT-States |             |             |             |             |             | UT-Zap50K   |             |             |             |             |             | C-GQA      |             |             |             |             |             |
|--------------|------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|------------|-------------|-------------|-------------|-------------|-------------|
|              | AUC        | HM          | S           | U           | s           | o           | AUC         | HM          | S           | U           | s           | o           | AUC        | HM          | S           | U           | s           | o           |
| CompCos [89] | 4.5        | 16.4        | 25.3        | 24.6        | 27.9        | 31.8        | 28.7        | 43.1        | 59.8        | 62.5        | 44.7        | 73.5        | -          | -           | -           | -           | -           | -           |
| CGE [4]      | 6.5        | 21.4        | 32.8        | 28.0        | 30.1        | 34.7        | 33.5        | 60.5        | 64.5        | 71.5        | 48.7        | 76.2        | 3.6        | 14.5        | 31.4        | 14.0        | 15.2        | 30.4        |
| CGE [4]*     | 6.3        | 20.0        | 31.6        | 27.3        | 30.3        | 34.5        | 31.5        | 46.5        | 60.3        | 64.5        | 46.3        | 74.4        | 3.7        | 14.9        | 30.8        | 14.7        | <b>15.8</b> | 29.0        |
| + BFV1       | <b>6.7</b> | <b>20.6</b> | <b>33.2</b> | <b>27.7</b> | <b>30.8</b> | <b>34.7</b> | <b>34.6</b> | <b>49.0</b> | <b>62.5</b> | <b>69.2</b> | <b>49.7</b> | <b>75.6</b> | <b>3.8</b> | <b>15.5</b> | <b>31.3</b> | <b>14.7</b> | <b>15.3</b> | <b>30.0</b> |

\* means we use the released code to reproduce the results. S means seen, U means unseen, s means state and o means object.

TABLE VIII  
ILLUSTRATION OF BATCHFORMERV1 FOR DOMAIN GENERALIZATION ON PACS [92]

| Method        | art_paint       | cartoon         | sketches        | photo           | Avg.        |
|---------------|-----------------|-----------------|-----------------|-----------------|-------------|
| Baseline      | 79.9±1.0        | 73.0±1.5        | 67.7±3.0        | 95.7±0.4        | 79.1        |
| + BFV1        | <b>80.4±0.2</b> | <b>73.8±2.0</b> | <b>68.6±1.8</b> | <b>96.3±0.2</b> | <b>79.8</b> |
| CORAL [98]    | 79.2±1.7        | 75.5±1.1        | 71.4±3.1        | 94.7±0.3        | 80.2        |
| + BFV1        | <b>80.6±0.9</b> | 74.7±1.9        | <b>73.1±0.3</b> | <b>95.1±0.3</b> | <b>80.9</b> |
| MixStyle [99] | 81.7±0.1        | <b>76.8±0.0</b> | 80.8±0.0        | 93.1±0.0        | 83.1        |
| + BFV1        | <b>84.8±0.4</b> | 75.3±0.0        | <b>81.1±0.4</b> | <b>93.6±0.0</b> | <b>83.7</b> |

TABLE IX  
ILLUSTRATION OF BATCHFORMERV1 FOR DOMAIN GENERALIZATION BASED ON SWAD [5]. THE BACKBONE IS RESNET-18

| Method   | PACS        | VLCS        | OfficeHome  | TerraIncognita |
|----------|-------------|-------------|-------------|----------------|
| SWAD [5] | 82.9        | 76.3        | 62.1        | 42.1           |
| + BFV1   | <b>83.7</b> | <b>76.9</b> | <b>64.3</b> | <b>44.8</b>    |

The backbone is ResNet-18.

domains and 65 classes; and 4) TerraIncognita [95] contains 24,788 images from 4 domains and 10 classes. For the baseline, we use ResNet-18 as the backbone. The model is then trained with the SGD optimizer for 30 epochs using an initial learning rate of 0.001 for the first 24 epochs and 0.0001 for the last 6 epochs. For fair comparison, we only use common data augmentations, e.g., flip, color-jitter, and scale. All models are trained for five times and the average performance is applied. When comparing with others methods, the implementations are based on [96]. For SWAD [5], we use the official code and always follow the same setting (see more details in the appendix).

**Results:** We show the experimental results on the domain generation in Tables VIII and IX. Specifically, Table VIII illustrates that BatchFormerV1 consistently improves the baseline methods: CORAL [97] and MixStyle [98]. In Table IX, we see that BatchFormerV1 clearly improves recent state-of-the-art method [5] on all four datasets. BatchFormerV1 improves [5] by over 2% on both OfficeHome and TerraIncognita, indicating that it can facilitate invariant representation learning to improve cross-domain generalization.

#### D. Contrastive Learning

Contrastive learning aims to learn representations that attract similar samples and dispel different samples, while BatchFormerV1 builds a transformer network among samples to implicitly explore sample relationships for representation learning. Therefore, BatchFormerV1 can also be easily applied to contrastive learning. We mainly evaluate BatchFormerV1 with MoCo-v2 [99] and MoCo-v3 [100] using linear classification protocol. We also show object detection results based on these pretrained backbone models in the appendix. As shown in

TABLE X  
ILLUSTRATION OF BATCHFORMERV1 FOR CONTRASTIVE LEARNING

| Method        | Epochs | Top-1       | Top-5 |
|---------------|--------|-------------|-------|
| MoCo-v2 [100] | 200    | 67.5        | -     |
| + BFV1        | 200    | <b>68.4</b> | 88.5  |
| MoCo-v3 [101] | 100    | 68.9        | -     |
| + BFV1        | 100    | <b>69.8</b> | 89.5  |

Table X, when using ResNet-50 as backbone, BatchFormerV1 can consistently improve self-supervised learning via contrastive learning, e.g., BatchFormerV1 improves both MoCo-v2 and MoCo-v3 by around 1% on ImageNet-1k.

#### E. Analysis

In this subsection, we provide some analyses of BatchFormerV1, including visual and gradient analyses.

**Visual Analysis:** We illustrate the visualized comparison with Grad-CAM [101] between the baseline and BatchFormerV1 in Fig. 8. We find BatchFormerV1 focuses on more details of objects and ignores spurious correlations. On the one hand, when the image includes complex scenes with many disturbing factors, BatchFormerV1 effectively improves the attention of the network on the corresponding object regions (e.g., the sea snake on the sandbeach, the dog on the snow and the insect on the leaf in Fig. 8). On the other hand, BatchFormerV1 also pays more attention on regions of the object when the scene is clear (e.g., the bird, dog, and spider). See more results in the appendix.

**Gradients Analysis:** BatchFormerV1 has increased new gradient backward: the loss of each label has the gradients on other images. In other words, we have implicitly augmented the samples for the class of each image in the mini-batches. The other images in the mini-batch can also be regarded as the virtual instances of current image class. The gradient is firmly related to the effect of each image label on other images. Fig. 9 illustrates the rarer the class is, the larger the gradients of the class have on other images in the mini-batch. Thus, BatchFormerV1 actually utilizes the other images to facilitate low-shot recognition via increasing gradients of few-shot labels on other images.

#### F. Image Classification

In this subsection, we evaluate BatchFormerV2 for image classification using vanilla vision transformer or ViT [22].

To compare with baseline methods, we follow the same training strategy with DeiT [102]. We perform image classification experiments on two popular datasets, CIFAR-100 [103] and ImageNet [104]. For CIFAR-100, we follow the setups in [71] and train all models with an initial learning rate 6e-4 and the batch size 1024. For ImageNet, we train all models for 300



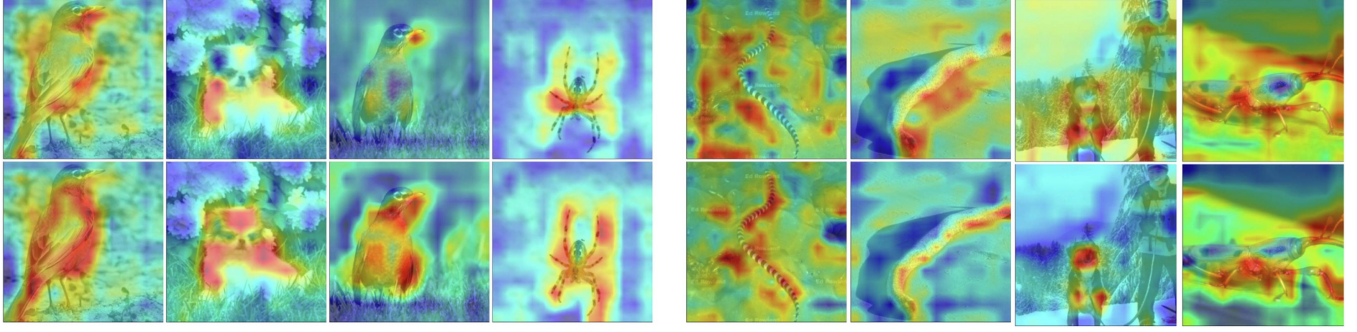


Fig. 8. Visualization of BatchFormerV1 on low-shot test images using Grad-Cam [82]. The first row is baseline, and the second row is BatchFormerV1. The left part of the figure shows that BatchFormerV1 enables the model to pay attention to more details when the scene is simple and clean, while the right part of the figure shows that BatchFormerV1 facilitates to ignore the spurious correlation in the image. More figures are shown in the appendix.

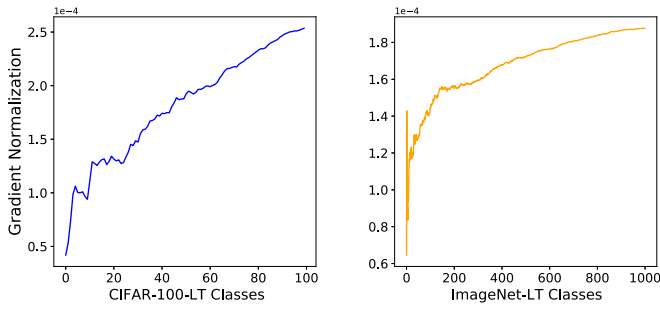


Fig. 9. The gradient of each class to other images in mini-batch on CIFAR-100-LT and ImageNet-LT (based on [82]). The classes are sorted by descending order according to the number of instances.

epochs with an initial learning rate  $1e-3$  and the batch size 1024. All experiments are conducted on a cluster with eight NVIDIA A100 GPUs (40 GB). When applying the proposed BatchFormerV2 module, we always use the same number of heads with the corresponding baseline model. For CIFAR-100, we insert BatchFormerV2 for all layers. For ImageNet, we insert BatchFormerV2 in the eighth layer. Empirically, we observe frequent crashes during training if we apply BatchFormerV2 in very early layers on ImageNet. More details and discussions are provided in the appendix.

**Results on CIFAR-100:** Current vision transformer architectures (e.g., ViT [22]) usually require large-scale training data or strong regularization to avoid the overfitting problem. Therefore, it still remains challenging to train vision transformers from scratch on small datasets. In this paper, we also find that the proposed BatchFormerV2 module can significantly improve the performance of vision Transformer on small datasets. As illustrated in Table XI, BatchFormerV2 significantly improves the performance of DeiT-B from 52.2% to 66.6% by **14.4%**, DeiT-S from 57.5% to 68.5% by **11%**, DeiT-Ti from 49.2% to 58.7% by **9.5%**. When training all models for more epochs, i.e., 300 epochs, the improvement remains considerable. The possible reason is BatchFormerV2 enables the information propagation among patches in different images, enhancing both optimization and generalization when working with small datasets. Additionally, we observe that DeiT-B does not outperform DeiT-S, possibly because DeiT-B is too large for a small dataset like CIFAR-100.

TABLE XI  
IMAGE CLASSIFICATION RESULTS ON CIFAR-100

| Model         | #Params | Input   | Epochs=100  | Epochs=300  |
|---------------|---------|---------|-------------|-------------|
| DeiT-Ti [103] | 5M      | $224^2$ | 49.2        | 69.2        |
| + BFV2        | 5M      | $224^2$ | <b>58.7</b> | <b>73.4</b> |
| DeiT-S [103]  | 22M     | $224^2$ | 57.5        | 72.5        |
| + BFV2        | 22M     | $224^2$ | <b>68.5</b> | <b>75.2</b> |
| DeiT-B [103]  | 86M     | $224^2$ | 52.2        | 71.8        |
| + BFV2        | 86M     | $224^2$ | <b>66.6</b> | <b>74.8</b> |

Specifically, following the experimental setups in [71], we train all models from scratch and report the top-1 accuracy (%).

TABLE XII  
IMAGE CLASSIFICATION RESULTS ON IMAGENET

| Model         | #Params | Input   | Top-1       | Top-5       |
|---------------|---------|---------|-------------|-------------|
| DeiT-Ti [103] | 5M      | $224^2$ | 72.2        | 91.1        |
| + BFV2        | 5M      | $224^2$ | <b>72.7</b> | <b>91.5</b> |
| DeiT-S [103]  | 22M     | $224^2$ | 79.8        | 95.0        |
| + BFV2        | 22M     | $224^2$ | <b>80.4</b> | <b>95.2</b> |
| DeiT-B [103]  | 86M     | $224^2$ | 81.7        | 95.5        |
| + BFV2        | 86M     | $224^2$ | <b>82.2</b> | <b>95.8</b> |

**Results on ImageNet:** Table XII shows that BatchFormerV2 consistently improves the performance among different ViT models. We observe BatchFormerV2 achieves similar improvement, i.e., around 0.5%, compared to the baseline. Compared to the improvement on object detection and panoptic segmentation, the improvement on image classification is relatively small. It might be because dense prediction requires to localize the objects in the images, i.e., there are multiple targets in the image, while image classification treats the whole image as the target and requires to recognize the image. Furthermore, we think the strong data augmentation in classification might be also a limitation for BatchFormerV2 on ImageNet.

### G. Object Detection

In this subsection, we evaluate BatchFormerV2 for common object detection using popular transformer-based object detectors such as DETR [62], Conditional-DETR [64], SMCA [65], and Deformable-DETR [63].

We perform all experiments on the most popular common object detection benchmark dataset, COCO 2017 [105], which contains 118 k training images and 5 k validation images.

TABLE XIII  
ILLUSTRATION OF BATCHFORMERV2 ON COMMON OBJECT DETECTION USING DETR [62] AND DEFORMABLE-DETR [63] AS BASELINES

| Method                   | Backbone   | $AP$        | $AP_{50}$   | $AP_{75}$   | $AP_S$      | $AP_M$      | $AP_L$      |
|--------------------------|------------|-------------|-------------|-------------|-------------|-------------|-------------|
| DETR [63]                | ResNet-50  | 34.8        | 55.6        | 35.8        | 14.0        | 37.2        | 54.6        |
| + BFV2                   | ResNet-50  | <b>36.9</b> | <b>57.9</b> | <b>38.5</b> | <b>15.6</b> | <b>40.0</b> | <b>55.9</b> |
| Conditional-DETR [65]    | ResNet-50  | 40.9        | 61.8        | 43.3        | 20.8        | 44.6        | 59.2        |
| + BFV2                   | ResNet-50  | <b>42.3</b> | <b>63.2</b> | <b>45.1</b> | <b>21.9</b> | <b>46.0</b> | <b>60.7</b> |
| SMCA (single scale) [66] | ResNet-50  | 41.0        | -           | -           | 21.9        | 44.3        | 59.1        |
| + BFV2                   | ResNet-50  | <b>42.3</b> | <b>63.5</b> | <b>45.4</b> | <b>22.5</b> | <b>45.7</b> | <b>60.1</b> |
| Deformable-DETR [64]     | ResNet-50  | 43.8        | 62.6        | 47.7        | 26.4        | 47.1        | 58.0        |
| + BFV2                   | ResNet-50  | <b>45.5</b> | <b>64.3</b> | <b>49.8</b> | <b>28.3</b> | <b>48.6</b> | <b>59.4</b> |
| Deformable-DETR* [64]    | ResNet-50  | 45.4        | 64.7        | 49.0        | 26.8        | 48.3        | 61.7        |
| + BFV2                   | ResNet-50  | <b>46.7</b> | <b>65.6</b> | <b>50.5</b> | <b>28.8</b> | <b>49.7</b> | <b>61.8</b> |
| Deformable-DETR [64]     | ResNet-101 | 44.5        | 63.7        | 48.3        | 25.8        | 48.6        | 59.6        |
| + BFV2                   | ResNet-101 | <b>46.0</b> | <b>65.2</b> | <b>50.5</b> | <b>28.4</b> | <b>49.8</b> | <b>60.7</b> |

\*indicates using iterative bounding box refinement.

During training, the backbone network is initialized from the weights pretrained on ImageNet-1K [104]. We run experiments on eight NVIDIA V100 GPUs (16 GB) for DETR, and eight NVIDIA A100 GPUs (40 GB) for Deformable DETR. If not otherwise stated, the batch-size for DETR, Conditional-DETR and SMCA is 16 and the default batch size for Deformable-DETR is 24. We insert the BatchFormerV2 module in the first transformer encoder layer in all experiments. The number of heads in BatchFormerv2 is 4. For fair comparison, all other hyperparameters follow the default configurations described in DETR [62], Conditional-DETR [64], SMCA [65] and Deformable-DETR [63].

**Results on COCO:** In Table XIII, BatchFormerV2 significantly improves the corresponding baseline methods. For example, without bells and whistles, BatchFormerV2 improves DETR by 2.1% and Deformable-DETR by 1.7% when using a ResNet-50 backbone. We observe consistent improvement on Conditional-DETR and SMCA. Moreover, we find that BatchFormerV2 mainly improves the object detection performance on small and medium objects. For example, BatchFormerV2 increases Deformable-DETR in  $AP_S$  by 1.9% and  $AP_M$  by 1.5%, respectively. For DETR, BatchFormerV2 increases  $AP_S$  and  $AP_M$  by 1.6% and 2.8%, respectively. We think via building Transformer along the pixel of the feature map in the batch dimension, BatchFormerV2 utilizes features from other images to facilitates object detection in current image. For small objects which is usually challenging to detect, BatchFormerV2 is able to incorporate objects from other images to detect (See visualization results in this section). Therefore, BatchFormerV2 significantly improves corresponding baselines. To better understand BatchFormerV2 for object detection, we also perform ablation studies on some key factors that may have influences on model performance as follows.

**Ablation Study on Batch Size:** Considering that BatchFormerV2 aims to learn sample relationships among each mini-batch during training, we evaluate the influence of different mini-batch size on BatchFormerV2 as follows. As shown in Table XIV, 1) when increasing the batch-size from 16 to 24, the performance can be further improved with a small margin; and 2) when further increasing the batch-size to 32, the performance is comparable, i.e., no additional improvements.

**Ablation Study on Insert Position:** We perform ablation studies on different insert positions of BatchFormerV2. Specifically,

TABLE XIV  
ABLATION STUDY ON BATCH SIZE

| Batch Size | Epochs | $AP$        | $AP_{50}$   | $AP_{75}$   | $AP_S$      | $AP_M$      | $AP_L$      |
|------------|--------|-------------|-------------|-------------|-------------|-------------|-------------|
| 16         | 50     | 44.7        | 63.5        | 48.9        | 27.3        | 48.1        | 59.1        |
| 24         | 50     | <b>45.1</b> | <b>64.1</b> | <b>49.3</b> | <b>28.5</b> | <b>48.4</b> | 59.4        |
| 32         | 50     | 44.9        | 63.8        | 48.8        | 27.7        | 48.3        | <b>60.0</b> |

Specifically, we use Deformable-DETR as our baseline and insert BatchFormerV2 module in the last transformer layer.

TABLE XV  
ABLATION STUDY ON INSERT POSITION

|      | L1-2 | L1-3 | L3-6 | L4-6 | L5-6 | L1          | L2   | L3   | L4   | L5   | L6   |
|------|------|------|------|------|------|-------------|------|------|------|------|------|
| $AP$ | 45.2 | 44.9 | 45.4 | 44.9 | 45.0 | <b>45.5</b> | 45.3 | 45.2 | 45.2 | 45.2 | 45.1 |

Specifically, "L1-3" indicates that we apply BatchFormerV2 modules from the first layer to the third layer.

TABLE XVI  
ABLATION STUDY ON SHARED MODULES

| Method          | $AP$        | $AP_{50}$   | $AP_{75}$   | $AP_S$      | $AP_M$      | $AP_L$      |
|-----------------|-------------|-------------|-------------|-------------|-------------|-------------|
| Deformable-DETR | 43.8        | 62.6        | 47.7        | 26.4        | 47.1        | 58.0        |
| + BFV2 (shared) | 44.9        | 63.6        | 49.1        | 27.7        | 47.9        | 59.6        |
| + BFV2          | <b>45.4</b> | <b>64.3</b> | <b>49.5</b> | <b>28.6</b> | <b>48.5</b> | <b>59.9</b> |

Specifically, here we apply BatchFormerV2 from the third to the sixth layers. All models are trained with 50 epochs.

we use Deformable-DETR as the baseline, which contains six transformer layers. As shown in Table XV, we find that: 1) the insert positions do have an important effect on the performance; and 2) Increasing the number of BatchFormerV2 modules does not always lead to improved object detection performance; and 3) Unlike in image classification experiments, where BatchFormerV2 is applied to the bottom layers, using BatchFormerV2 in the top layers appears to be more effective for dense prediction tasks. This aligns with the semantic hierarchy seen in deep representation learning for computer vision: image classification requires high-level semantics while it is better mine low-level semantic relationships for object detection.

**Ablation Study on Shared Modules:** We evaluate whether sharing all BatchFormerV2 modules among different layers will benefit dense prediction tasks. As shown in Table XVI, we find that, without sharing the modules, it could bring 0.5% improvement compared to the shared scheme. It may suggest that the dense sample relationships are varying among different layers/levels, which also explains that, in different visual recognition tasks, BatchFormerV2 may need to be added into different layers. Furthermore, the training overhead of BatchFormerV2



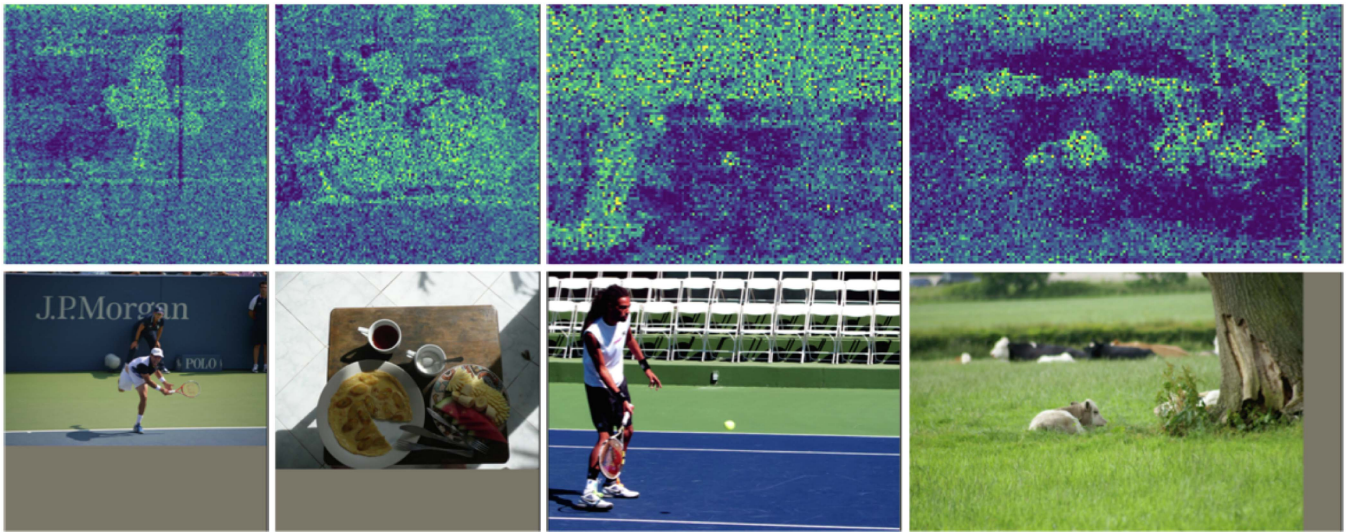


Fig. 10. Visualization of the attention maps. We apply BatchFormerV2 in the first transformer layer in Deformable-DETR.

TABLE XVII  
PANOPTIC SEGMENTATION WITH DETR [62] ON THE COCO VAL DATASET

| Method    | $PQ$        | $SQ$        | $RQ$        | $PQ^{th}$   | $SQ^{th}$   | $RQ^{th}$   | $PQ^{st}$   | $SQ^{st}$   | $RQ^{st}$   | $AP$        |
|-----------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|
| DETR [63] | 43.4        | 79.3        | 53.8        | 48.2        | 79.8        | 59.5        | 36.3        | 78.5        | 45.3        | 31.1        |
| + BFV2    | <b>45.1</b> | <b>80.3</b> | <b>55.3</b> | <b>50.5</b> | <b>81.1</b> | <b>61.5</b> | <b>37.1</b> | <b>79.1</b> | <b>46.0</b> | <b>33.4</b> |

is relatively small compared to the backbone, though it varies based on factors such as batch size and insertion positions. For image classification, the overall training overhead increases by approximately 9%. In contrast, for object detection and segmentation, the training overhead increases by approximately 20%, depending on the batch size and the specific configuration of BatchFormerV2 within the backbone network.

*Visualization:* BatchFormerV2 is applied into transformer encoder layers along each spatial position, and it enables the information propagation among samples in a mini-batch via attention mechanism. Here, we visualize the attention of each slot on other slots along with the same position among the batch dimension. Specifically, we show the feature map in the first transformer encoder layer of Deformable-DETR. Fig. 10 shows that BatchFormerV2 mainly focuses on the objects (e.g., person, chairs), while paying less attention on the background (e.g., play ground, grass). This actually demonstrates that BatchFormerV2 improves the object localization via the attention among different samples. This is also consistent to the panoptic segmentation results in Section IV-H, where the improvements on the *things* categories are significant than those on the *stuff* categories. In addition, we observe that self-attention highlights all the regions of the objects with clear boundaries.

#### H. Panoptic Segmentation

In this subsection, we evaluate BatchFormerV2 for panoptic segmentation, i.e., a combination of instance and semantic segmentation, on the MS-COCO dataset. We use the panoptic annotation in [106], which contains additional 53 *stuff* categories in addition to 80 *things* categories from the original MS-COCO

dataset. We use DETR [62] as our baseline, i.e., we utilize a mask head to generate panoptic segmentation results for both *stuff* and *things* classes in a unified way [106]. Following [62], we first train the model with BatchFormerV2 for object detection to predict bounding boxes around *stuff* and *things* classes 300 epochs. We then finetune the new mask head for extra 25 epochs.

*Results:* We report the panoptic quality ( $PQ$ ) and the breakdown performances on things ( $PQ^{th}$ ) and stuff ( $PQ^{st}$ ) in Table XVII. Specifically, we observe that BatchFormerV2 significantly improves  $AP$  by 2.3% and  $PQ$  by 1.7%. We also notice the improvement on  $PQ^{th}$  is much larger than  $PQ^{st}$ . That is, BatchFormerV2 improves  $PQ^{th}$  by 2.3%, while the improvement on  $PQ^{st}$  is only 0.8%. This result is consistent with the results of object detection: by enabling the information propagation, BatchFormerV2 mainly facilitates object detection and instance segmentation. Furthermore, following [62], we actually freeze the bounding box branch and transformer layers (include BatchFormerV2) when finetuning the mask head, we find that the performance of panoptic segmentation is also significantly improved. A possible explanation is that BatchFormerV2 improves the optimization of the backbone and the transformer encoder for better object detail modeling for bounding box detection and subsequently facilitates the segmentation performance when finetuning the mask head.

*Visualization:* To better understand how BatchFormerV2 helps dense prediction, we provide visualization results in Fig. 11. Specifically, BatchFormerV2 mainly improves the segmentation details and the small object segmentation. For example, the segmentation boundaries of the door have been significantly improved, while the baseline model mistakenly segments the door as the wall in Fig. 11. Meanwhile, the legs



Fig. 11. A visual comparison between DETR [62] with and without BatchFormerV2. Specifically, the first row is the original image, the second row is the result without BatchFormerV2, and the last row is the result with BatchFormerV2.

of the desk are more clear with the help of BatchFormerV2. In the second column, the segmentation details of airplane become better with BatchFormerV2, e.g., the tail and front wheels. In the last column, the baseline model ignores the segmentation of the grass (i.e., small stuffs), while it can correctly segment the grass with BatchFormerV2.

## V. CONCLUSION

In this paper, we propose equipping deep neural networks with the ability to explore sample relationships within each training mini-batch. The proposed BatchFormerV1 allows gradient propagation from each label to all images in the mini-batch, which can be intuitively interpreted as virtual sample augmentation. To further capture multi-scale sample relationships for dense prediction, we extend this approach to BatchFormerV2. Leveraging a two-stream training pipeline, the proposed module serves as a plug-and-play component that can be seamlessly integrated into various vision transformers without incurring additional inference costs. We evaluate the module through extensive experiments on over ten datasets. The results demonstrate significant improvements in: 1) data-scarcity scenarios, including long-tailed recognition, zero-shot learning, domain generalization, and contrastive learning; and 2) various visual recognition tasks, such as image classification, object detection, and panoptic segmentation.

## REFERENCES

- [1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
- [2] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum contrast for unsupervised visual representation learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 9726–9735.
- [3] X. Wang, L. Lian, Z. Miao, Z. Liu, and S. X. Yu, "Long-tailed recognition by routing diverse distribution-aware experts," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–12.
- [4] M. Naeem, Y. Xian, F. Tombari, and Z. Akata, "Learning graph embeddings for compositional zero-shot learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 953–962.
- [5] J. Cha et al., "Swad: Domain generalization by seeking flat minima," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 22405–22418.
- [6] S. Shankar, V. Piratla, S. Chakrabarti, S. Chaudhuri, P. Jyothi, and S. Sarawagi, "Generalizing across domains via cross-gradient training," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–12.
- [7] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "Mixup: Beyond empirical risk minimization," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–13.
- [8] S. Li, K. Gong, C. H. Liu, Y. Wang, F. Qiao, and X. Cheng, "MetaSAug: Meta semantic augmentation for long-tailed visual recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 5212–5221.
- [9] G. Ghiasi et al., "Simple copy-paste is a strong data augmentation method for instance segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 2918–2928.
- [10] Y.-X. Wang, D. Ramanan, and M. Hebert, "Learning to model the tail," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 7032–7042.
- [11] J. Wang, T. Lukasiewicz, X. Hu, J. Cai, and Z. Xu, "RSG: A simple but effective module for learning imbalanced datasets," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 3784–3793.
- [12] L. Zhu and Y. Yang, "Inflated episodic memory with region self-attention for long-tailed visual recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 4344–4353.
- [13] K. Kato, Y. Li, and A. Gupta, "Compositional learning for human object interaction," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 234–251.
- [14] Z. Hou, X. Peng, Y. Qiao, and D. Tao, "Visual compositional learning for human-object interaction detection," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 584–600.
- [15] Y. Atzmon, F. Kreuk, U. Shalit, and G. Chechik, "A causal view of compositional zero-shot recognition," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 5212–5221.
- [16] Z. Liu, Z. Miao, X. Zhan, J. Wang, B. Gong, and S. X. Yu, "Large-scale long-tailed recognition in an open world," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 2537–2546.
- [17] Y. Xian, C. H. Lampert, B. Schiele, and Z. Akata, "Zero-shot learning—A comprehensive evaluation of the good, the bad and the ugly," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 9, pp. 2251–2265, Sep. 2019.



- [18] X. Peng, Z. Huang, X. Sun, and K. Saenko, "Domain agnostic learning with disentangled representations," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 5102–5112.
- [19] M. Arjovsky, L. Bottou, I. Gulrajani, and D. Lopez-Paz, "Invariant risk minimization," 2019, *arXiv: 1907.02893*.
- [20] Z. Huang, H. Wang, E. P. Xing, and D. Huang, "Self-challenging improves cross-domain generalization," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 124–140.
- [21] A. Vaswani et al., "Attention is all you need," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.
- [22] A. Dosovitskiy et al., "An image is worth 16×16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 1–12.
- [23] Z. Liu et al., "Swin transformer: Hierarchical vision transformer using shifted windows," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2021, pp. 9992–10002.
- [24] L. Yuan et al., "Tokens-to-token ViT: Training vision transformers from scratch on imageNet," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2021, pp. 558–567.
- [25] X. Chu et al., "Twins: Revisiting the design of spatial attention in vision transformers," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 9355–9366.
- [26] C.-F. Chen, R. Panda, and Q. Fan, "Regionvit: Regional-to-local attention for vision transformers," in *Proc. Int. Conf. Learn. Representations*, 2022, pp. 1–14.
- [27] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 448–456.
- [28] Z. Hou, B. Yu, and D. Tao, "Batchformer: Learning to explore sample relationships for robust representation learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 7246–7256.
- [29] Z. Hou, B. Yu, C. Wang, Y. Zhan, and D. Tao, "Batchformerv2: Exploring sample relationships for dense representation learning," 2022, *arXiv:2204.01254*.
- [30] Y. Liu et al., "Learning to propagate labels: Transductive propagation network for few-shot learning," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–11.
- [31] R. Hou, H. Chang, B. Ma, S. Shan, and X. Chen, "Cross attention network for few-shot classification," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 4005–4016.
- [32] A. K. Mondal, V. Jain, and K. Siddiqi, "Mini-batch graphs for robust image classification," 2021, *arXiv:2105.03237*.
- [33] P. Tokmakov, Y.-X. Wang, and M. Hebert, "Learning compositional representations for few-shot recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 6372–6381.
- [34] J. He, A. Kortylewski, and A. Yuille, "Compas: Representation learning with compositional part sharing for few-shot classification," 2021, *arXiv:2101.11878*.
- [35] J. Kossen, N. Band, C. Lyle, A. N. Gomez, T. Rainforth, and Y. Gal, "Self-attention between datapoints: Going beyond individual input-output pairs in deep learning," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 28742–28756.
- [36] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "Smote: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.
- [37] H. Han, W.-Y. Wang, and B.-H. Mao, "Borderline-SMOTE: A new over-sampling method in imbalanced data sets learning," in *Proc. Int. Conf. Adv. Intell. Comput.*, 2005, pp. 878–887.
- [38] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 9, pp. 1263–1284, Sep. 2009.
- [39] J. Byrd and Z. Lipton, "What is the effect of importance weighting in deep learning?," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 872–881.
- [40] Y. Cui, M. Jia, T.-Y. Lin, Y. Song, and S. Belongie, "Class-balanced loss based on effective number of samples," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 9268–9277.
- [41] J. Tan et al., "Equalization loss for long-tailed object recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 11662–11671.
- [42] Z. Zhong, J. Cui, S. Liu, and J. Jia, "Improving calibration for long-tailed recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 16489–16498.
- [43] M. A. Jamal, M. Brown, M.-H. Yang, L. Wang, and B. Gong, "Rethinking class-balanced methods for long-tailed visual recognition from a domain adaptation perspective," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 7610–7619.
- [44] B. Zhou, Q. Cui, X.-S. Wei, and Z.-M. Chen, "BBN: Bilateral-branch network with cumulative learning for long-tailed visual recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 9719–9728.
- [45] J. Cai, Y. Wang, and J.-N. Hwang, "ACE: Ally complementary experts for solving long-tailed recognition in one-shot," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2021, pp. 112–121.
- [46] Y.-Y. He, J. Wu, and X.-S. Wei, "Distilling virtual examples for long-tailed recognition," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2021, pp. 235–244.
- [47] G. Blanchard, G. Lee, and C. Scott, "Generalizing from several related classification tasks to a new unlabeled sample," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2011, pp. 2178–2186.
- [48] K. Zhou, Z. Liu, Y. Qiao, T. Xiang, and C. C. Loy, "Domain generalization: A survey," 2021, *arXiv:2103.02503*.
- [49] J. Wang, C. Lan, C. Liu, Y. Ouyang, W. Zeng, and T. Qin, "Generalizing to unseen domains: A survey on domain generalization," 2021, *arXiv:2103.03097*.
- [50] C. H. Lampert, H. Nickisch, and S. Harmeling, "Learning to detect unseen object classes by between-class attribute transfer," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2009, pp. 951–958.
- [51] M. M. Palatucci, D. A. Pomerleau, G. E. Hinton, and T. Mitchell, "Zero-shot learning with semantic output codes," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2009.
- [52] W. Wang, V. W. Zheng, H. Yu, and C. Miao, "A survey of zero-shot learning: Settings, methods, and applications," *ACM Trans. Intell. Syst. Technol.*, vol. 10, no. 2, pp. 1–37, 2019.
- [53] Y. Zhu, M. Elhoseiny, B. Liu, X. Peng, and A. Elgammal, "A generative adversarial approach for zero-shot learning from noisy texts," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 1004–1013.
- [54] K. Zhou, Y. Yang, T. Hospedales, and T. Xiang, "Learning to generate novel domains for domain generalization," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 561–578.
- [55] Y. Balaji, S. Sankaranarayanan, and R. Chellappa, "Metareg: Towards domain generalization using meta-regularization," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 998–1008.
- [56] Y. Zhao et al., "Learning to generalize unseen domains via memory-based multi-source meta-learning for person re-identification," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 6277–6286.
- [57] P. Chattopadhyay, Y. Balaji, and J. Hoffman, "Learning to balance specificity and invariance for in and out of domain generalization," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 301–318.
- [58] V. Piratla, P. Netrapalli, and S. Sarawagi, "Efficient domain generalization via common-specific low-rank decomposition," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 7728–7738.
- [59] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," 2014, *arXiv:1409.0473*.
- [60] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv: 1810.04805*.
- [61] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–12.
- [62] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 213–229.
- [63] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, "Deformable detr: Deformable transformers for end-to-end object detection," 2020, *arXiv: 2010.04159*.
- [64] D. Meng et al., "Conditional DETR for fast training convergence," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2021, pp. 3631–3640.
- [65] P. Gao, M. Zheng, X. Wang, J. Dai, and H. Li, "Fast convergence of DETR with spatially modulated co-attention," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2021, pp. 3621–3630.
- [66] B. Cheng, A. Schwing, and A. Kirillov, "Per-pixel classification is not all you need for semantic segmentation," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 17864–17875.
- [67] S. Zheng et al., "Rethinking semantic segmentation from a sequence-to-sequence perspective with transformers," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 6877–6886.
- [68] R. Strudel, R. Garcia, I. Laptev, and C. Schmid, "Segmenter: Transformer for semantic segmentation," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2021, pp. 7242–7252.
- [69] W. Wang et al., "Pyramid vision transformer: A versatile backbone for dense prediction without convolutions," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2021, pp. 568–578.

- [70] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo, "SegFormer: Simple and efficient design for semantic segmentation with transformers," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 12077–12090.
- [71] X. Chen, S. Xie, and K. He, "An empirical study of training self-supervised vision transformers," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2021, pp. 9640–9649.
- [72] H. Bao, L. Dong, and F. Wei, "Beit: BERT pre-training of image transformers," 2021, *arXiv:2106.08254*.
- [73] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," 2021, *arXiv:2111.06377*.
- [74] A. Radford et al., "Learning transferable visual models from natural language supervision," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 8748–8763.
- [75] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum contrast for unsupervised visual representation learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 9729–9738.
- [76] A. Ramesh et al., "Zero-shot text-to-image generation," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 8821–8831.
- [77] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 7132–7141.
- [78] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 8026–8037.
- [79] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 3431–3440.
- [80] Y. LeCun et al., "Convolutional networks for images, speech, and time series," *The Handbook Brain Theory Neural Netw.*, vol. 3361, no. 10, 1995, Art. no. 1995.
- [81] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 7132–7141.
- [82] J. Ren et al., "Balanced meta-softmax for long-tailed visual recognition," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 4175–4186.
- [83] J. Cui, Z. Zhong, S. Liu, B. Yu, and J. Jia, "Parametric contrastive learning," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2021, pp. 695–704.
- [84] G. Van Horn et al., "The inaturalist species classification and detection dataset," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 8769–8778.
- [85] B. Zhou, A. Lapedriza, A. Khosla, A. Oliva, and A. Torralba, "Places: A 10 million image database for scene recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 6, pp. 1452–1464, Jun. 2018.
- [86] L. Xiang, G. Ding, and J. Han, "Learning from multiple experts: Self-paced knowledge distillation for long-tailed classification," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 247–263.
- [87] B. Kang et al., "Decoupling representation and classifier for long-tailed recognition," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 1–11.
- [88] M. Mancini, M. F. Naeem, Y. Xian, and Z. Akata, "Open world compositional zero-shot learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 5222–5230.
- [89] P. Isola, J. J. Lim, and E. H. Adelson, "Discovering states and transformations in image collections," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 1383–1391.
- [90] A. Yu and K. Grauman, "Fine-grained visual comparisons with local learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2014, pp. 192–199.
- [91] S. Purushwalkam, M. Nickel, A. Gupta, and M. Ranzato, "Task-driven modular networks for zero-shot compositional learning," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2019, pp. 3593–3602.
- [92] D. Li, Y. Yang, Y.-Z. Song, and T. M. Hospedales, "Deeper, broader and artier domain generalization," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 5542–5550.
- [93] C. Fang, Y. Xu, and D. N. Rockmore, "Unbiased metric learning: On the utilization of multiple datasets and web images for softening bias," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2013, pp. 1657–1664.
- [94] H. Venkateswara, J. Eusebio, S. Chakraborty, and S. Panchanathan, "Deep hashing network for unsupervised domain adaptation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 5018–5027.
- [95] S. Beery, G. Van Horn, and P. Perona, "Recognition in terra incognita," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 456–473.
- [96] J. Jiang, B. Chen, B. Fu, and M. Long, "Transfer-learning-library," 2020. [Online]. Available: <https://github.com/thuml/Transfer-Learning-Library>
- [97] B. Sun and K. Saenko, "Deep coral: Correlation alignment for deep domain adaptation," in *Proc. Eur. Conf. Comput. Vis.*, 2016, pp. 443–450.
- [98] K. Zhou, Y. Yang, Y. Qiao, and T. Xiang, "Domain generalization with mixstyle," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–12.
- [99] X. Chen, H. Fan, R. Girshick, and K. He, "Improved baselines with momentum contrastive learning," 2020, *arXiv: 2003.04297*.
- [100] X. Chen, S. Xie, and K. He, "An empirical study of training self-supervised vision transformers," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 9620–9629.
- [101] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 618–626.
- [102] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jegou, "Training data-efficient image transformers & distillation through attention," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 10347–10357.
- [103] A. Krizhevsky et al., "Learning multiple layers of features from tiny images," *Tech. Rep.*, pp. 1–47, 2009.
- [104] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2009, pp. 248–255.
- [105] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," in *Proc. Eur. Conf. Comput. Vis.*, 2014, pp. 740–755.
- [106] A. Kirillov, K. He, R. Girshick, C. Rother, and P. Dollár, "Panoptic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 9404–9413.
- [107] P. Wang, K. Han, X.-S. Wei, L. Zhang, and L. Wang, "Contrastive learning based hybrid networks for long-tailed image classification," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 943–952.
- [108] C. Wah, S. Branson, P. Welinder, P. Perona, and S. Belongie, "The Caltech-UCSD birds-200-2011 dataset," *Tech. Rep.*, pp. 1–8, 2011.
- [109] Y. Shen, J. Qin, L. Huang, L. Liu, F. Zhu, and L. Shao, "Invertible zero-shot recognition flows," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 614–631.
- [110] S. Narayan, A. Gupta, F. S. Khan, C. G. Snoek, and L. Shao, "Latent embedding feedback and discriminative features for zero-shot classification," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 479–495.
- [111] M. Everingham, L. Van Gool, C. K. Williams, J. Winn, and A. Zisserman, "The pascal visual object classes (VOC) challenge," *Int. J. Comput. Vis.*, vol. 88, no. 2, pp. 303–338, 2010.
- [112] Y. Wu, A. Kirillov, F. Massa, W.-Y. Lo, and R. Girshick, "Detectron2," 2019. [Online]. Available: <https://github.com/facebookresearch/detectron2>
- [113] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jegou, "Training data-efficient image transformers & distillation through attention," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 10347–10357.
- [114] R. Wightman, "PyTorch image models," 2019. [Online]. Available: <https://github.com/rwightman/pytorch-image-models>
- [115] D. Krueger et al., "Out-of-distribution generalization via risk extrapolation (rex)," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 5815–5826.
- [116] X. Peng, B. Usman, N. Kaushik, J. Hoffman, D. Wang, and K. Saenko, "VisDA: The visual domain adaptation challenge," pp. 1–10, 2017.
- [117] Y. Zhang, T. Liu, M. Long, and M. Jordan, "Bridging theory and algorithm for domain adaptation," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 7404–7413.
- [118] C. Zimmermann, D. Ceylan, J. Yang, B. Russell, M. Argus, and T. Brox, "Freihand: A dataset for markerless capture of hand pose and shape from single RGB images," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2019, pp. 813–822.
- [119] K. Lin, L. Wang, and Z. Liu, "Mesh graphormer," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2021, pp. 12919–12928.
- [120] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, "Cutmix: Regularization strategy to train strong classifiers with localizable features," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2019, pp. 6023–6032.



**Zhi Hou** received the BE degree in computer science from the South China University of Technology, China. He is currently working toward the PhD degree with the School of Computer Science, University of Sydney, NSW, Australia. His research interests include computer vision and deep learning. His publications are mainly in top computer vision conferences, such as CVPR and ECCV.



**Baosheng Yu** received the BE degree from the University of Science and Technology of China (USTC), China, in 2014, and the PhD degree from the University of Sydney (USYD), Australia, in 2019. He is currently an assistant professor with Nanyang Technological University (NTU), Singapore. His research focuses on machine learning and deep learning, with applications in computer vision and medical image analysis. He has published more than 50 papers in leading conferences and journals, including CVPR, ICCV, ECCV, and *IEEE Transactions on Pattern Analysis and Machine Intelligence*.



MM, AAAI, *International Journal of Computer Vision*, and *IEEE Transactions on Multimedia*.

**Yibing Zhan** received the BE and the PhD degrees from the University of Science and Technology of China, in 2012 and 2018, respectively. From 2018 to 2020, he was an associate researcher with the Computer and Software School, Hangzhou Dianzi University. He is currently an algorithm scientist with the JD Explore Academy. His research interest include graphical models and multimodal learning, including cross-modal retrieval, scene graph generation, and graph neural networks. He has publications on various top conferences and journals, such as CVPR, ACM



**Chaoyue Wang** received the bachelor degree from Tianjin University (TJU), China, and the PhD degree from the University of Technology Sydney (UTS), Australia. He is a research scientist with JD Explore Academy (JD.com). Before that, he was a postdoctoral researcher in Machine Learning and Computer Vision with the School of Computer Science, University of Sydney (Usyd). His research outcomes have been published in prestigious journals and prominent conferences, such as *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *IEEE Transactions on Evolutionary Computation*, *IEEE Transactions on Image Processing*, NeurIPS, CVPR, IJCAI. He received the Distinguished Student Paper Award in the 2017 International Joint Conference on Artificial Intelligence (IJCAI-17).



**Dacheng Tao** (Fellow, IEEE) is currently a distinguished University professor with the College of Computing & Data Science, Nanyang Technological University. He mainly applies statistics and mathematics to artificial intelligence and data science, and his research is detailed in one monograph and more than 200 publications in prestigious journals and proceedings with leading conferences, with best paper awards, best student paper awards, and test-of-time awards. His publications have been cited more than 120 K times and he has an h-index more than 170 in Google Scholar. He received the 2015 and 2020 Australian Eureka Prize, the 2018 IEEE ICDM Research Contributions Award, the 2019 Diploma of The Polish Neural Network Society, and the 2021 IEEE Computer Society McCluskey Technical Achievement Award. He is a fellow of the Australian Academy of Science, AAAS, and ACM.